# Learning Verified Monitors for Hidden Markov Models

Luko van der Maas and Sebastian Junges

Radboud University, Nijmegen, the Netherlands

**Abstract.** Runtime monitors for hidden Markov models obtain a sequence of observations and must classify whether the system is safe or unsafe. That is, the probability of being in a dangerous state conditioned on these observations exceeds some acceptable threshold. The key contribution of this paper is the first correct-by-construction synthesis method for such monitors, represented as finite state machines. First, we establish the coNP-hardness of checking whether a finite state machine is indeed a correct monitor, i.e., a monitor without misclassifications. Second, we provide a reduction that reformulates the search for misclassifications into a standard probabilistic system synthesis problem. Third, we integrate the verification routine into an active automata learning routine to synthesise correct monitors. Fourth, we provide a prototypical implementation that shows the feasibility and limitations of the approach on a series of benchmarks.

## 1 Introduction

Runtime assurance is an essential ingredient in the deployment of safe autonomous systems. Runtime monitors provide assurance by flagging potentially dangerous system behavior by observing a system execution. More precisely, a monitor receives a stream of observations about the system and outputs a verdict, e.g., it raises an alarm that the system has left some safety envolope. A monitor is correct, if it correctly raises such alarms based on a formal specification. Various challenges in creating correct runtime monitors for (semi-)autonomous systems have been identified [**empty citation**], such as: (1) the state of the system is only partially observable, i.e., the stream of observations comes from sensor readings and does uniquely identify the state of a system, (2) the behavior of the system and/or the sensors may be subject to probabilistic uncertainty, (3) the monitor itself is subject to resource constraints (time, memory, etc), and (4) the monitor is itself safety-critical and should therefore be subject to extensive validation. Challenges (1,2) can be (partially) addressed by modelling the system as a hidden Markov model (HMM), Challenges (3,4) can be (partially) addressed by representing a monitor as, e.g., a (small) finite state machine. Concretely, this paper focusses on the following question: *Is a given finite state machine a correct monitor for a given HMM?* The paper studies the complexity of this problem, provides a practical verification procedure, and embeds this verification procedure into a learning framework.

*What are HMMs?* In this paper, we assume that the system including its sensors is adequately described as a discrete Hidden Markov Model (HMM). Markov chains (MCs) describe system behavior subject to probabilistic uncertainty. Paths through an MC are sequences of states that describe system executions. HMMs extend MCs by labelling their states with colours (aka *observations*). Intuitively, the observations can be used to model the information that the monitor receives. In HMMs, every path can be lifted to a sequence of observations, which we call a *trace*. The trace associated to a system execution describes the information received by the monitor.

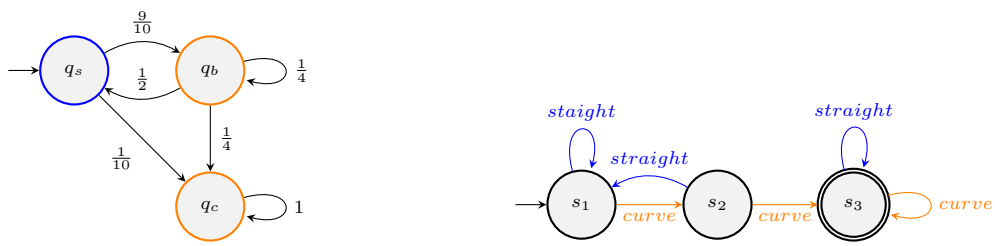
*Risk of a trace.* In runtime monitoring, an essential task is to estimate whether the current system is dangerous. In the simplest setting, a specification describes a set of states is dangerous. The risk of a trace is then the probability of ending in a dangerous state, conditioned on the fact that the system execution matches the trace. For a given trace, we may compute this risk, e.g., either via a reduction to model checking [**empty citation**] or by a (forward) filtering that tracks a distribution over the current states [**empty citation**].

*What are correct monitors?* Often, we are not interested in this precise risk, but rather want to partition the set of horizon-bounded traces into safe and unsafe traces: Unsafe traces are traces whose risk does exceed some fixed *acceptable threshold*, all other traces are safe. Indeed, a *correct* monitor obtains a trace and raises an alarm iff a trace is not safe. The key verification problem in this paper asks whether a monitor, represented as a deterministic finite automaton, accepts (i.e., raises an alarm) exactly the unsafe traces. We note that in this paper, we only consider traces that are bounded by some fixed horizon.

*Illustrative toy example.* SJ: TODO make more concrete, add Fig. As an running example, we consider an oversimplified car driving scenario, loosely inspired by runtime monitors obtained from high-fidelity simulations [**empty citation**]. A car can be in three states: It can be on a dry road, on an icy patch, or it can drifted off the road. The HMM in Figure 3 describes the situation, where a car alternates between dry and icy road segments, and where being on a icy (dry) segment positively affects the probability that the next segment is icy (dry). When on an icy road, there is a higher probability to drift off the road. Our sensor can detect dry roads, but cannot distinguish between icy roads and off-road conditions. We can define the trace risk as the probability to be off-road, conditioned on a a trace, such as $dry \cdot ice \cdot ice$. In **??**, we depict a finite automaton that describes the set of traces that end with two consecutive icy patches. The key verification question in this paper is now: Is this a correct monitor for threshold 0.9? Put differently, does it raise an alarm iff the trace risk is above 0.9.

SJ: Do we also want a small figure of a system feeding observations into a monitor?

*Novel complexity results.* Deciding whether a monitor is correct is coNP-complete (Theorem 1). In particular, we study the dual problem that asks whether a monitor misclassifies at least one trace. We consider a simpler variant of this problem that asks whether there is a bounded trace in the HMM that is unsafe.




(a) Example HMM, where the orange states $q_b$ and $q_c$ have the observation *bend* and the blue state $q_s$ has the observation *straight*.

(b) An example monitor for the HMM in fig. 1a, which raises an alarm as soon as two steps in a curve have been observed.

Fig. 1: An HMM and a correct monitor that raises an alarm if the probability of being off-road is above 0.8.

[1] The problem is clearly in NP (assuming that the bound is unary encoded). We show that this simplified problem is also strongly NP-hard (Lemma 1) and the corresponding optimization problem, that asks to approximate the risk of the trace with the heighest risk, is APX-hard (Lemma 2). Intuitively, APX-hardness means that the existence of an efficient approximation algorithm would have major complexity-theoretic implications.

*Our verification approach.* While deciding whether a monitor is correct is thus in general not tractable, we suggest to utilise recent advances in the synthesis of probabilistic systems. We concentrate on proving the absence of missed alarms, i.e., we concentrate on showing that the monitor correctly identifies every unsafe trace. A similar reduction works for showing the absence of spurious alarm. First, for a single trace, computing the risk can be reduced to computing a reachability probability in a MC that is intuitively a kind of product between a DFA that accepts exactly the trace and the HMM [**empty citation**]. Thus, it is natural to reduce our problem to the question: Is there a DFA (accepting exactly one trace, accepted by our monitor) such that the probability in the product MC exceeds a threshold. If the answer is no, then the monitor has no missed alarms. This decision problem can be solved using (exact) probabilistic model synthesis methods [**empty citation**] as implemented in the tool Paynt [**empty citation**][2]. We discuss our verification approach in Section 3.

*Our learning approach.* Ultimately, we do not only want to verify the correctness of a monitor, but we want to construct such a monitor. Above, we already mentioned that the monitors are formal languages. When considering bounded trace,

[1] The results also apply to checking for safe traces.

[2] Alternatively, in Section **??**, we show that we can construct a product of an acyclic DFA accepting sets of traces and the HMM. However, this yields partially observable MDP, where we must search for memoryless policies, similar to a construction in [4]. This problem is equivalent to the synthesis problem and can therefore also be solved using Paynt [1].

TODO add our framework

Fig. 2

these monitors are regular, and can thus be captured using DFAs. Inspired by a recent short paper [**empty citation**], we investigate learning these monitors: We choose to do this using active automata learning [3, 8], which closely relates to other popular oracle-guided inductive synthesis loops [**empty citation**]. To use active automata learning (AAL), we must provide a *membership oracle* that decides for a single trace whether the risk of the trace exceeds a threshold and an *equivalence oracle* that decides whether a given hypothesis monitor is indeed correct. The membership oracle can be implemented by computing the risk for a single trace as above and comparing it to a threshold, while the equivalence oracle can be implemented by checking for the absence of missed and false alarms. Practically, some minor modifications are necessary to embed our approach in AAL framework: We have to handle the finite horizon, resolve some traces that are not relevant for the monitor, and we combine conformance and equivalence queries to boost performance.

*Putting it together.* In Fig. 2, we outline our proposed framework to learn a monitor. SJ: TODO explain framework

*Empirical evaluation.* SJ: TODO summarise results of the empirical eval here.

*Contributions.* In summary, this paper provides a first framework to learn runtime monitors — encoded as finite automata — that are verified to be correct with respect to a given HMM. The main contributions are as follows: (1) We study monitor verification and show complexity-theoretic hardness of the underlying problem, even in an approximative setting. (2) We reduce the monitor verification to probabilistic model synthesis problems that exhaustively search for a counterexample. (3) We learn monitors by using the verifier above to answer equivalence queries in conjunction with active automata learning. (4) We demonstrate the feasibility and limitations of both the verifier and the learner on a series of benchmarks. Our prototype can handle HMMs with thousands of states, up to a hundred observations, and monitors with over 100 states.

## 2 Problem statement

In this section, we recap standard notions on stochastic models and finite automata and introduce the formal problem statement.

*Notation.* Let $X$ be a finite set. A distribution $\mu$ over $X$ is a mapping $\mu\colon X \to [0,1]$ such that $\sum_{x \in X} \mu(x) = 1$. The set of all distributions over $X$ is written $\Delta(X)$.

### 2.1 Models

In this section, we briefly introduce MDPs, which are important later, and HMMs which are crucial to the problem statement. Details can be found in [5].

**Definition 1 (MDP).** *A* Markov decision process *(MDP) is a tuple* $\mathcal{M} = (S, \iota, Act, \mathbf{P})$ *with a countable nonempty set* $S$ *of* states*, the* initial state $\iota \in S$*, the* partial transition function $\mathbf{P} \colon S \times Act \rightharpoonup \Delta(S)$*.*

We use $Act(s) = \{a \mid \mathbf{P}(s, a) \neq \bot\}$ as the set of enabled actions. We assume that there are no deadlocks, i.e., for every $s \in S$, $Act(s) \neq \emptyset$. A path $\pi$ in an MDP is a sequence $s_0 a_0 s_1 \ldots \in (S \times Act)^* \times S$, such that $a_i \in Act(s_i)$ and $P(s_i, a_i)(s_{i+1}) > 0$ for every $i \geq 0$. The set of (finite) paths is denoted by $\Pi^{\mathcal{M}}_{fin}$ and the set of paths of at most length $h$ are $\Pi^{\mathcal{M}}_{h}$. If the MDP is clear from the context, the $\mathcal{M}$ is dropped.

A *Markov chain* (MC) is an MDP where $|Act(s)| = 1$ for every state $s \in S$. We simplify notation and write MCs also as tuple $(S, \iota, \mathbf{P})$ and write $\mathbf{P}(s)$ to refer to the unique distribution $\mathbf{P}(s, a)$. Paths in a MC can be simplified to be sequences of (only) states.

The probability measure $Pr^{\mathcal{M}}$ of an MC $\mathcal{M}$ is the unique probability measure following from the canonical $\sigma$-algebra associate with $mc$.

Condition probability on traces defined using Paths

**Definition 2 (HMMs).** *A (risk-labelled) HMM is a tuple* $(S, \iota, \mathbf{P}, Z, obs, r)$ *such that* $(S, \iota, \mathbf{P})$ *is a MC,* $Z$ *is a finite set of observations, and* $obs \colon S \to Z$ *is the (deterministic) observation function*[3]*, and* $r \colon S \to \mathbb{R}_{\geq 0}$*,*

Notions such as paths are lifted from MCs. Furthermore, traces $\tau$ are sequences of observations, and the set of traces of a HMM is the language $\mathcal{L}(\mathcal{M})$ of the HMM. The set $\text{Paths}(\tau)$ contains all paths that correspond to a trace $\tau$, $\text{Paths}(\tau) = \{\pi \in \Pi^{fin} \mid obs(\pi) = \tau\}$. SJ: Also needs things like the probability of a trace.

*Example 1.*

**Definition 3.** *A deterministic finite automaton (DFA) is a tuple* $\mathcal{A} = (Q, \Sigma, \delta, \iota, F)$*.* $Q$ *is a finite set of states,* $\Sigma$ *is an alphabet,* $\delta \colon Q \times \Sigma \rightharpoonup Q$ *is a (partial) transition function,* $\iota$ *is the initial state, and* $F \subseteq Q$ *is a set of accept (or: final) states.*

Let $\varepsilon$ denote the empty word. We lift the transition relation to words, which is defined recursively: $\delta^*(q, \varepsilon) = \varepsilon$ if $a \in \Sigma$ and $\delta^*(q, w \cdot a) = \delta(\delta^*(q, w), a)$.

The *language* $\mathcal{L}(\mathcal{A})$ of a DFA $\mathcal{A}$ is the set of all words accepted by the $\mathcal{A}$, $\mathcal{L}(\mathcal{A}) = \{w \in \Sigma^* \mid \delta^*(\iota, w) \in F\}$.

### 2.2 Formal Problem Statement

The monitor verification problem consists of two sub problems. The no false alarms problem, and the no missed alarms problem. Both of these sub problems

[3] We use determinstic observation functions for concise definitions. Non-deterministic observation functions can be expressed via a blowup of the HMM, see e.g. [**empty citation**]

entail finding the trace with either the minimum or maximum risk. We will discuss the missed alarm problem in the following sections. In section 3.5 we will adapt to the no false alarms problem.

The no missed alarms problem entails ensuring that all traces from the MC, which are not in the monitor, have at most a risk of $\lambda$.

**Definition 4 (Risk of a trace).** *Given a HMM $\mathcal{M}$. The risk of trace $\tau \in \mathcal{L}(\mathcal{M})$ is:*

$$R_r(\tau) := \sum_{\pi \in \Pi^h_{\mathcal{M}}} Pr(\pi \mid \tau) \cdot r(\pi_{\downarrow}).$$

The risk of a trace can be used to partition the set of all traces.

**Definition 5 (Safe traces/Unsafe traces).** *Given a HMM $\mathcal{M}$, a* safe threshold *$\lambda_s \in \mathbb{R}$ and an* unsafe threshold *$\lambda_u \in \mathbb{R}$ with $\lambda_s \leq \lambda_u$. A trace $\tau \in \mathcal{L}(\mathcal{M})$ with $R(\tau) > \lambda_u$ is an* unsafe trace *(for $\mathcal{M}$ and $\lambda_u$). Conversely, trace $\tau$ is a* safe trace*, if $R(\tau) < \lambda_s$ (for $\mathcal{M}$ and $\lambda_u$).*

In the definition above, we deliberately do not require $\lambda_s = \lambda_u$. In particular, by picking $\lambda_s < \lambda_u$, some traces are neither safe nor unsafe. We write $\mathbb{S}^{\leq h}_{\mathcal{M},\lambda_s}$ to denote the set of safe traces of length at most $h$ and $\mathbb{U}^{\leq h}_{\mathcal{M},\lambda_u}$ for the set of unsafe traces of length at most $h$. We often omit the sub/superscripts.

*Example 2.*

We are now ready to define correct monitors.

**Definition 6 (Monitor).** *A DFA $\mathcal{A}$ is a monitor for HMM $\mathcal{M}$ if the alphabet for $\mathcal{A}$ coincides with the observations in $\mathcal{M}$.*

**Definition 7 (Missed/False alarms).** *Given a monitor $\mathcal{A}$ for HMM $\mathcal{M}$, a horizon $h$, and thresholds $\lambda_s \leq \lambda_u \in \mathbb{R}$, the set of* missed alarms *is $\mathsf{mA}^{\leq h}_{\mathcal{M},\lambda_s}(\mathcal{A}) := \mathbb{U}^{\leq h}_{\mathcal{M},\lambda_u} \setminus \mathcal{L}(\mathcal{A})$. The set of* false alarms *is $\mathsf{fA}^{\leq h}_{\mathcal{M},\lambda_s}(\mathcal{A}) := \mathbb{S}^{\leq h}_{\mathcal{M},\lambda_s} \cap \mathcal{L}(\mathcal{A})$.*

We drop the sub/superscripts whenever possible. In terms of a classification problem, missed alarms can be seen as false negatives while false alarms are false positives.

**Definition 8 (Correct monitor).** *Given thresholds $\lambda_s, \lambda_u$ and horizon $h$, a monitor $\mathcal{A}$ for $\mathcal{M}$ is* correct *if $\mathsf{mA}^{\leq h}_{\mathcal{M},\lambda_s}(\mathcal{A}) = \emptyset = \mathsf{fA}^{\leq h}_{\mathcal{M},\lambda_u}(\mathcal{A})$.*

Equivalently, a monitor is correct if it raises an alarm for every unsafe trace and for no safe trace:

**Corollary 1 (Correct monitor).** *A monitor $\mathcal{A}$ for HMM $\mathcal{M}$ with $\mathbb{U}^{\leq h}_{\mathcal{M},\lambda_u} \subseteq \mathcal{L}(\mathcal{A}) \subseteq \Sigma^* \setminus \mathbb{S}^{\leq h}_{\mathcal{M},\lambda_s}$ is a correct monitor.*

Fig. 3: Central example to the paper

**Problem statements.** This paper tackles the following problem statements: Given HMM $\mathcal{M}$, thresholds $\lambda_s, \lambda_u$ and horizon $h$,

1. Given monitor $\mathcal{A}$ for $\mathcal{M}$, are there missed alarms, i.e., is $\mathsf{mA}^{\leq h}_{\mathcal{M},\lambda_s}(\mathcal{A}) = \emptyset$?
2. Given monitor $\mathcal{A}$ for $\mathcal{M}$, are there false alarms, i.e., is $\mathsf{fA}^{\leq h}_{\mathcal{M},\lambda_u}(\mathcal{A}) = \emptyset$?
3. Find a correct monitor $\mathcal{A}$ for $\mathcal{M}$ w.r.t. $\lambda_s, \lambda_u$ and $h$.

Problem statements 1 and 2 together allow checking whether a monitor is correct.

*Remark 1.* A correct monitor must exist, as $\mathbb{U}^{\leq h}$ is finite and thus regular.

We discuss our solution to the no missed alarms problem in section 3, we adapt the solution to the no false alarm problem in section 3.5.

*Central example* We now introduce the central example used in this paper. And explain how the monitor verification problem and verified monitor learning problem apply to it.

*Example 3.* Given the risk function assigning 1 to $q_c$ and 0 to all other states, the traces $\tau_1 = straight, corner, corner$ and $\tau_2 = straight, corner$ have risks $\frac{19}{40}, \frac{4}{40}$ respectively. If $\lambda$ is $\frac{1}{4}$ and the horizon $h$ is 3, $\tau_2$ is the trace with maximum risk not in the monitor. And its risk is below $\lambda$. Thus the monitor does not have an missed alarms.

Include verified monitor learning problem

## 3 Monitor verification

In this section we will discuss our solution to the monitor verification problem. We focuss on the no missed alarms problem, and adapt our solution to the no false alarms problem in section 3.5. We start by translating the problem to the Conditional trace probability (CTP) problem in section 3.1, by incorporating the monitor in the HMM. Next, in section 3.3, we reduce the CTP problem to a consisitent policy synthesis problem for MDPs. Lastly, we show how to solve this consistent policy synthesis problem using PAYNT[2] in section 3.4.

### 3.1 CTP problem

We will solve the Monitor verification problem by looking at the simpler *Conditional trace probability* (CTP) problem. We will show a reduction from the Co-Monitor Verification Problem to CTP.

**Definition 9 (CTP Decision Problem).** *Given an HMM $\mathcal{M}$ with colors $C$, a unary encoded horizon $H$, a set of goal states $G$, and a threshold $\lambda \in (0, 1]$, is there a trace $\tau \in C^H$ such that $\sum_{\pi \in \Pi^{\mathcal{M}}_H | \pi_\downarrow \in G} Pr^{\mathcal{M}}(\pi \mid \tau) \cdot r(\pi_\downarrow) \geq \lambda$.*

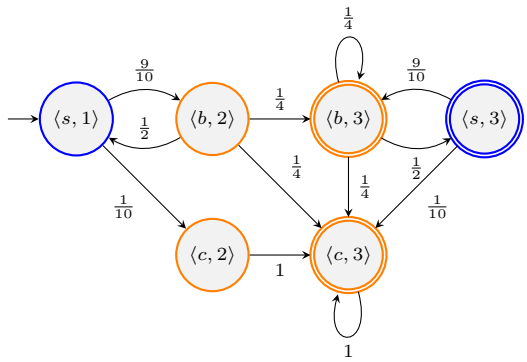



Fig. 4: The central example HMM transformed into the CTP problem, using the Product Markov Chain construction [5]. States are named with their HMM state, $\{s, b, c\}$ and their monitor state, $\{1, 2, 3\}$. States with the accepting label are marked as accepting states.

As we will be shown in section 5 the CTP problem is NP-complete.

The CTP problem differs from the monitor verification problem in two important aspects. First, the CTP problem no longer considers the monitor and the risk. Second, instead of deciding all traces are below a threshold, we have to find a trace above a threshold. Finding a trace similar to the dual of the Monitor Verification Problem, the *co-Monitor Verification Problem*.

**Definition 10 (co-Monitor Verification Problem).** *Given a HMM $\mathcal{M}$ with a state risk function $r : S \to \mathbb{R}_{\geq 0}$, a DAG monitor DFA $\mathcal{A}$ with as alphabet the observations of $\mathcal{M}$, a risk thresholds $\lambda \in [0, \inf)$, and a horizon $h \in \mathbb{N}^+$. Decide $R_r^{\max}(\overline{\mathcal{A}}) < \lambda$, where the* weighted maximum risk function $R_r^{\max} \colon DFA \to \mathbb{R}_{\geq 0}$ *is defined as*

$$R_r^{\max}(\mathcal{A}) := \max_{\tau \in \mathcal{L}(\mathcal{M}) \cap \mathcal{L}(\mathcal{A})} \sum_{\pi \in \Pi_{\mathcal{M}}^h} Pr(\pi \mid \tau) \cdot r(\pi_{\downarrow})$$

fix def

Finding a trace in the co-MV problem disproves the MV problem for the same inputs.

*Reduction to CTP* In order to translate the co-MV problem to the CTP problem we combine the monitor and the HMM using the baierPrinciplesModelChecking2008. The monitor has to be complete, however, this can be ensured without loss of generality. Our central example transformed into a CTP decision problem can be found in fig. 4.

Maybe say complement of the monitor because missed alarms

### 3.2 Asyclic CTP problem

We further simplify the monitor verification problem by incorporating the horizon, the goal states and the risk into the HMM. This results in the acyclic CTP (ACTP) problem.

**Definition 11 (Asyclic CTP problem).** *Given an acyclic HMM $\mathcal{M}$ with colors $C$, an alarm state $a$, and a threshold $\lambda \in (0,1]$, is there a trace $\tau \in C^\star$ such that $\sum_{\pi \in \Pi^{\mathcal{M}}_{H-1} \in G} Pr^{\mathcal{M}}(\pi \cdot a \mid \tau) \geq \lambda$.*

Explain problem

*Reduction* Given an instance of the CTP problem with HMM $\mathcal{M}^{\text{CTP}} = (S, \mathbf{P}, \iota, AP, L, obs, C)$, good states $G$, horizon $H$, risk function $r$, and threshold $\lambda$, we reduce the CTP problem to an ACTP problem.

We unroll $\mathcal{M}^{\text{CTP}}$ over the horizon $H$, creating the unrolled HMM $\mathcal{M}^{\text{unrolledCTP}} = (S \times \mathbb{N}^{+\leq H}, \mathbf{P}', \iota', AP, L', obs', C)$. $\mathbf{P}'$ is defined as

get better name for tussenstap hmm

$$\mathbf{P}'(\langle s, i\rangle, \langle q, i+1\rangle) = \mathbf{P}(s, q)$$

for $i \in \mathbb{N}^{+<H}$. Horizon states do not have any outgoing transitions. $\iota'$, $L'$, and $obs'$ are similarly unrolled versions of $\iota$, $L$, and $obs$.

Now, we incorporate the risk and good states by adding new states, and new observations. We define the input HMM for the ACTP problem as $\mathcal{M}^{\text{ACTP}} = (S \times \mathbb{N}^{+\leq H} \cup \{alarm, safe, ignore\}, \mathbf{P}'', \iota', AP, L', obs'', C \cup \{ignore, end\})$. We normalize the risk function $r$ as $r'$, such that all values fall in $(0,1)$ in order to use it as a transition probability in $\mathbf{P}''$

$$\mathbf{P}'(\langle s, H\rangle, alarm) = \begin{cases} r'(s) & \text{if } s \in G \\ 0 & \text{otherwise} \end{cases}$$

$$\mathbf{P}'(\langle s, H\rangle, safe) = \begin{cases} 1 - r'(s) & \text{if } s \in G \\ 0 & \text{otherwise} \end{cases}$$

$$\mathbf{P}'(\langle s, H\rangle, ignore) = \begin{cases} 1 & \text{if } s \notin G \\ 0 & \text{otherwise} \end{cases}$$

Both the *alarm* and *safe* sates get the *end* observation. The *ignore* state gets the *ignore* observation.

This reduction applied to the example $\mathcal{M}^{\text{CTP}}$ can be seen in fig. 5

### 3.3 Reduction to Consistent Policy Synthesis

We will reduce the CTP problem into a consistent policy synthesis problem on Markov decision processes (MDPs). An MDP is a automata where every state contains actions, which have probabilistic outcomes.

**Definition 12 (MDPs).** *A Markov decision process is a tuple $\mathcal{M} = (S, Act, \mathbf{P}, \iota, AP, L)$ where, $S$ is a countable set of states, $Act$ is a set of actions, $\mathbf{P} : S \times Act \times S \to [0,1]$ is the transition probability function, $\iota$ is the initial state, $AP$ is a set of atomic propositions, and $L : S \to 2^{AP}$ is a labeling function. From [5]*

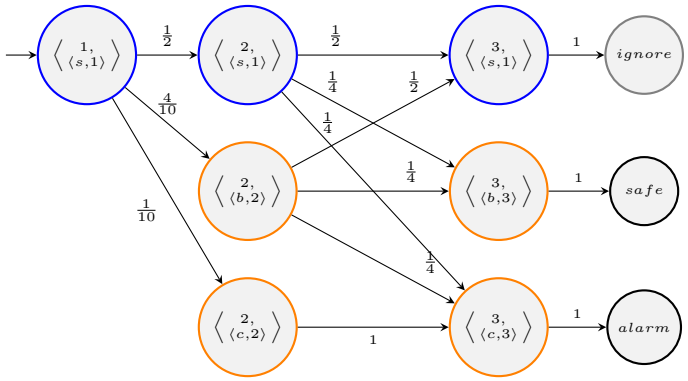



Fig. 5: ACTP, blue and orange are straight and corner observations. Black is the new end observation, and gray is the new ignore observation.

A policy for an MDP is a function $\sigma \colon \Pi^{\mathcal{M}} \to Act$, which given a path through the MDP, gives a action. Given a policy, a MDP can be induced into a MC. Policy synthesis for an MDP for a property $\phi$ thus entails finding a policy for an MDP such that the induced MC entails $\phi$.

The central idea beding using MDPs to solve the ACTP problem is letting a policy in the MDP be equivalent to a trace in the ACTP. However, to enusure this equivalence we will need to limit our policies. To do this, we introduce a form of consistency.

**Definition 13.** *A consistent policy $\sigma$ is a policy which has the following property with $\iota$ the observation of the initial state and $e$ is the end observation*

$$\exists_{\tau \in C^\star} \ \forall_{\pi \in Paths(\sigma)} \ obs(\pi) = (\iota\tau^<)^+ \iota\tau e^\star$$

*Thus, the trace of a path obtained using $\sigma$ is a finite sequence of initial observations and prefixes of $\tau$ finished by observing the initial state, fully $\tau$ and an infinite amount of end observations.*

Technically there are no observations in the MDP, we could rewrite in terms of the actions taken by the MDP, but might be less intuitive

However, we are not able to enforce this form of consisitency in our policies. Instead, we look at semi-consistent schedulers.

**Definition 14.** *A semi-consistent policy $\sigma$ is a policy which has the following property*

$$\exists_{\tau \in C^\star} \ \forall_{\pi \in Paths(\sigma)} \ obs(\pi) = (\iota\tau^<)^+ (\iota\tau e^\star \mid (\iota\tau^<)^\star)$$

*Thus, a semi-consistent scheduler is a consistent scheduler where the induced trace never has to observe an end state.*

This is quite a lot of non-intuitive notation, how could I do it differently

We claim that the maximal semi-consistent policy for reachability of the *goal* or *stop* states will always be a consisitent policy if an end transition can be taken.

Give proof?

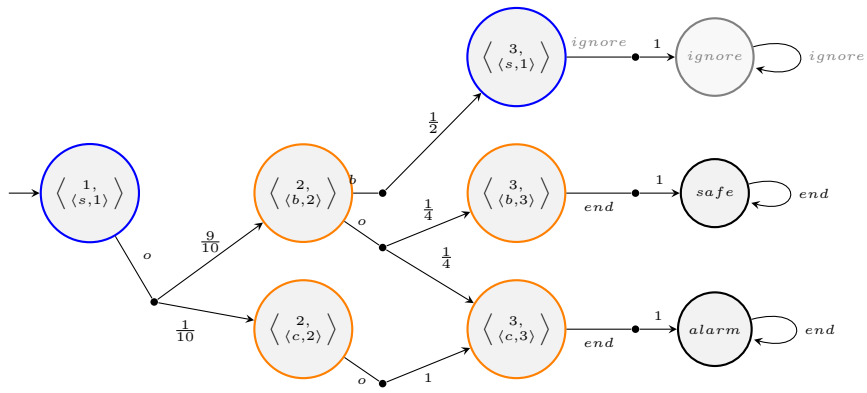



Fig. 6: The central example transformed into an mdp to be used as the model in the MDP policy synthesis problem. Every state has an action which contains only transitions to states which had the observation related to that action.

bad explanation

*Reduction* We will reduce the ACTP problem with an HMM $\mathcal{M}^{\text{ACTP}} = (S \times \mathbb{N}^{+\leq H} \cup \{alarm, safe, ignore\}, \mathbf{P}'', \iota', AP, L', obs'', C \cup \{ignore, end\})$ and a threshold $\lambda$ into an MDP such that a consistent policy coincides with a trace in $\mathcal{M}^{\text{ACTP}}$.

Intuitively, we will let the MDP choose its next observation. Any probability not going towards this observation will be redirected to the initial state, thus resetting the path.

We construct the policy synthesis MDP $\mathcal{M} = (S \times \mathbb{N}^{<H}, C \cup \{ignore, end\}, \mathbf{P}''', \iota'', \{alarm, safe\}, L'')$ where $\mathbf{P}'''$ is definded as

$$\mathbf{P}'''(\langle s, i\rangle, c, \langle q, i+1\rangle) := \begin{cases} \mathbf{P}''(\langle s, i\rangle, \langle q, i+1\rangle) & \text{if } obs(q) = c \\ 1 - \sum_{q' \in S \mid obs(q')=c} P^{\mathcal{M}}(s, q') & \text{if } q = \iota \\ 0 & \text{otherwise} \end{cases}$$

### 3.4 Performing policy synthesis using memoryless policies

### 3.5 Adapting to no false alarms

## 4 Learning Monitors

## 5 Complexity

**Theorem 1.** *Checking whether a monitor is correct with respect to a unary encoded horizon is coNP-complete.*

In fact, we study in this section the dual to this problem:

**Definition 15.**

NP-membership follows for the monitor co-verification as false alarms or a missed alarm (of length up to horizon) are witnesses. Verifying whether this trace is indeed a false alarm can be done in polynomial time by first checking whether the automaton accepts it and then computing the trace risk and comparing it to the thresholds. Each of these steps can be done in polynomial time. SJ: Maybe this result is so crucial that we should explain somewhere how this works in polynomial time.

For hardness, we show NP-hardness of the CTP problem from **??**. As a solution to the monitor co-verification problem suffices to solve the CTP problem, this implies NP harndess of the former.

**Lemma 1.** *The CTP Decision Problem is NP-hard.*

SJ: What about the probability of a trace being zero?

The result holds even for $\lambda = 1$ and acyclic HMMs. We reduce the problem from 3CNF. We outline the construction below.

Consider a 3CNF formula $\varphi = \bigwedge c_1 \dots c_m$ over variables $X$, $|X| = n$, with clause $c_j = \ell_j^1 \vee \ell_j^2 \vee \ell_j^3$ and each literal $\ell_j^i \in \{x, \neg x \mid x \in X\}$. We transform this into a CTP instance with $\lambda = 1$ and an acyclic HMM $\mathcal{M}_\varphi$ with observations $Z = \{\#, \bot, \top\}$. The only state with positive risk is a state $t$. By construction, there is a trace with risk 1 iff there is a satisfying assignment to $\varphi$. Note that there is a trace with risk 1 iff there is a trace where all corresponding paths end in state $t$.

Before we give a formal definition of $\mathcal{M}_\varphi$, we give some intuition, supported by Fig. 7. We represent assignments $\alpha \colon X \to \{0, 1\}$ by traces through the HMM $\#\# \cdot \alpha(x_0) \cdot \# \cdot \alpha(x_1) \cdot \dots \cdot \#$. This trace matches with exactly $m$ paths through the HMM, one for each clause. That is, intuitively, $\mathcal{M}_\varphi$ randomly selects a clause $c_j$ with probability $1/m$ in the initial state and then 'evaluates' $c_j$ with respect to an assignment $\alpha$, by starting in $s_{1,j}$.

That is, a path through clause $c_j$ 'reads' variable $x_i$ in state $s_{i,j}$ and transitions to $s_{i,j}^\top$ and $s_{i,j}^\bot$, both with probability $1/2$. State $s_{i,j}$, $s_{i,j}^\top$ and $s_{i,j}^\bot$ have observations $\#, \top, \bot$, respectively. Thus, the conditional probability to reach $s_{i,j}^\top$ is $1/m$ if $\alpha(x_i) = \top$ and 0 otherwise. Then, if the literal $x_i$ (or $\neg x_i$) occurs in clause $c_j$, we transition from $s_{i,j}^\top$ (or $s_{i,j}^\bot$, resp.) to $s'_{i+1,j}$, otherwise, we transition to $s_{i+1,j}$. The idea is that (conditionally) reaching state $s'_{i,j}$ with positive cconditional probability means that the partial assignment $\alpha(x_1), \dots, \alpha(x_i)$ satisfies clause $c_j$, while reaching $s_{i,j}$ with positive conditional probability means that the clause is either unresolved or unsatisfied given the partial assignment. Thus, $t$

We construct the following HMM $\mathcal{M} = (S, \mathbf{P}, \iota, AP, L)$ and set $\lambda = 1$:

- $S = \{s_{i,j}, s_{i,j}^\bot, s_{i,j}^\top, s'_{i,j}, s'^{\bot}_{i,j}, s'^{\top}_{i,j} \mid i \in \{1, \dots, n+1\}, j \in \{1, \dots, m\}\} \cup \{s_\iota\}$.
- $\mathbf{P}$ is given such that for all $i \in \{1, \dots, n\}, j \in \{1, \dots, m\}$:
  - $\mathbf{P}(s_\iota, s_{1,j}) = \frac{1}{m}$,
  - $\mathbf{P}(s_{i,j}, s_{i,j}^\top) = \mathbf{P}(s_{i,j}, s_{i,j}^\bot) = \frac{1}{2} = \mathbf{P}(s'_{i,j}, s'^{\top}_{i,j}) = \mathbf{P}(s'_{i,j}, s'^{\bot}_{i,j})$,

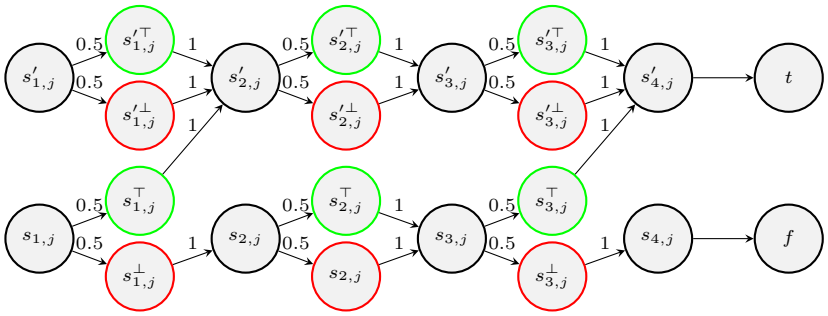



Fig. 7: The main gadget for the reduction of Lemma 1, demonstrated for the clause $x_1 \vee \neg x_3$ and variables $x_1, x_2, x_3$.

- $\mathbf{P}(s^{\top}_{i,j}, s'_{i+1,j}) = [x_i = \ell^k_j$ for some k$]$, $\mathbf{P}(s^{\top}_{i,j}, s_{i+1,j}) = [x_i \neq \ell^k_j$ for all k$]$,
- $\mathbf{P}(s^{\bot}_{i,j}, s'_{i+1,j}) = [\neg x_i = \ell^k_j$ for some k$]$, $\mathbf{P}(s^{\bot}_{i,j}, s_{i+1,j}) = [\neg x_i \neq \ell^k_j$ for all k$]$
- $\mathbf{P}(s'_{n+1,j}, t) = 1 = \mathbf{P}(s_{n+1,j}, f)$

– $c(s^{\top}_{i,j}) = (s'^{\top}_{i,j}) = \top$, $c(s^{\bot}_{i,j}) = (s'^{\bot}_{i,j}) = \bot$, $c(s_{i,j}) = c(s'_{i,j}) = \# = c(s_\iota) = c(t) = c(f)$.

–

**Approximability** Approximating the verification of monitors is also hard, in the sense that it is hard to approximate the maximal risk that a monitor admits.

The maximal risk problem is an optimization problem

**Definition 16 (CTP Optimization Problem).**

**Lemma 2.** *The CTP Optimization Problem is APX-hard.*

We show this by means of an L-reduction from MAX-3SAT, which is an inapproximable and APX-hard problem [**empty citation**]. The construction follows along the reduction for the NP-hardness.

## 6 Experiments

We build our tool in the python programming language on top of the model checker Storm [6] and the POMDP model checker PAYNT [2]. This tool includes both the ToVer verifier algorithm and the learning algorithm employing ToVer for equality queries.

TODO: Write further description of the tool.

*Benchmarks* We evaluate both the ToVer verifier and the learning algorithm. We evaluate the ToVer verifier using the following questions:

$\mathbf{Q}^1_{VM}$ How does the runtime of ToVer verifier depend on its parameters?
$\mathbf{Q}^2_{VM}$ How much does the ToVer verification depend on PAYNT?

$\mathbf{Q}^3_{VM}$ Which PAYNT strategy is the most efficient in the ToVer benchmarks?

Maybe not a full question?

We also evaluate the learning algorithm by looking at the following questions:

$\mathbf{Q}^1_L$ Does ToVer learning guarantee a better bound then learning using a sampling EQ?

$\mathbf{Q}^2_L$ How do the ToVer learning monitors and sampling learning monitors compare in size?

$\mathbf{Q}^3_L$ Do monitors learned without a horizon limit in MQ generalize to outside the horizon?

Again, maybe less relevant?

## 7 Related

- LTLf
- Predictors or so by Ziemek et al.
- MDPs as Distribution transformers

## 8 Conclusion and Future Work

- Long traces (complexity and practical implementation).
- Unbounded traces (It woudl be great if we can add why this is hard)

## References

[1] Roman Andriushchenko et al. "Inductive synthesis of finite-state controllers for POMDPs". In: *UAI*. Vol. 180. Proceedings of Machine Learning Research. PMLR, 2022, pp. 85–95.

[2] Roman Andriushchenko et al. "PAYNT: A Tool for Inductive Synthesis of Probabilistic Programs". In: *Computer Aided Verification*. Ed. by Alexandra Silva and K. Rustan M. Leino. Cham: Springer International Publishing, 2021, pp. 856–869. ISBN: 978-3-030-81685-8. DOI: 10.1007/978-3-030-81685-8_40.

[3] Dana Angluin. "Learning Regular Sets from Queries and Counterexamples". In: *Inf. Comput.* 75.2 (1987), pp. 87–106.

[4] Thom S. Badings et al. "CTMCs with Imprecisely Timed Observations". In: *TACAS (2)*. Vol. 14571. Lecture Notes in Computer Science. Springer, 2024, pp. 258–278.

[5] Christel Baier and Joost-Pieter Katoen. *Principles of Model Checking*. MIT Press, 2008. ISBN: 978-0-262-02649-9.

[6] Christian Hensel et al. "The Probabilistic Model Checker Storm". In: *International Journal on Software Tools for Technology Transfer* 24.4 (Aug. 1, 2022), pp. 589–610. ISSN: 1433-2787. DOI: 10.1007/s10009-021-00633-z. URL: https://doi.org/10.1007/s10009-021-00633-z (visited on 09/02/2024).

[7] Orna Lichtenstein, Amir Pnueli, and Lenore Zuck. "The Glory of the Past". In: *Logics of Programs*. Ed. by Rohit Parikh. Berlin, Heidelberg: Springer, 1985, pp. 196–218. ISBN: 978-3-540-39527-0. DOI: 10.1007/3-540-15648-8_16.

[8] Frits W. Vaandrager. "Model learning". In: *Commun. ACM* 60.2 (2017), pp. 86–95.

# A Alternative formulations of the problem

Instead of giving a specification DFA, we could instead give a TL specification [7] which is evaluated for every state. Since TL has both future and past operators both now reachability, $\diamondminus\, l$, and soon reachability, $\bigcirc^n \diamondminus l$, for a label $l$ can be expressed. While using a specification DFA can only express now reachability.

The only impact this reformulation of the problem would have is in step 1. Here, instead of taking the product MC, we check if the above described TL formula has a probability above a certain threshold. If that is the case we mark it with the $\circledcirc_{\mathcal{A}}$ label. This MC is then used for the subsequent steps.

# B Reformulation with monitors

## B.1 Benchmarks

# C Problem Statements

We have two problems we want to solve. The no false alarms problem and the no missed alarms problem. They have slightly different problem statements and therefore also slightly different solutions.

## C.1 No false alarms

All traces $\tau$ from the MC which are in the monitor have at least a probability $p \in I$ of reaching the good state.

**Definition 17.** *Given a Markov chain $\mathcal{M} = (S, \mathbf{P}, \iota, AP, L)$ with observations in the set of colors $C$, a specification DFA $\mathcal{A} = \big(Q_{\mathcal{A}}, 2^{AP}, \delta_{\mathcal{A}}, \iota_{\mathcal{A}}, F_{\mathcal{A}}\big)$, $AP$, and a DAG monitor DFA $\mathcal{B} = (Q_{\mathcal{B}}, C, \delta_{\mathcal{B}}, \iota_{\mathcal{B}}, F_{\mathcal{B}})$. For all traces $\tau$ from the MC and in the monitor, the probability of $\pi \in \mathcal{L}(\mathcal{A})$ given that $\pi$ conforms to $\tau$ is in a given interval $I$*

$$\operatorname*{in}^{I}_{\tau \in \mathcal{L}(\mathcal{M}) \cap \mathcal{L}(\mathcal{B})} Pr^{\mathcal{M}}(\pi \in \mathcal{L}(\mathcal{A}) \mid \pi \in \mathrm{Cyl}(\mathrm{Paths}(\tau)))$$

### C.2 No missed alarms

All traces $\tau$ which reach the good state with probability $p > t$, are in the monitor. By taking the contrapositive, we can instead show that all traces $\tau$ which are not in the monitor reach the good state with probability $p \leq t$.

**Definition 18.** *Given a Markov chain $\mathcal{M} = (S, \mathbf{P}, \iota, AP, L)$ with observations in the set of colors $C$, a specification DFA $\mathcal{A} = \left(Q_{\mathcal{A}}, 2^{AP}, \delta_{\mathcal{A}}, \iota_{\mathcal{A}}, F_{\mathcal{A}}\right)$, $AP$, and a DAG monitor DFA $\mathcal{B} = (Q_{\mathcal{B}}, C, \delta_{\mathcal{B}}, \iota_{\mathcal{B}}, F_{\mathcal{B}})$. For all traces $\tau$ from the MC and not in the monitor, the probability of $\pi \in \mathcal{L}(\mathcal{A})$ given that $\pi$ conforms to $\tau$ is in a given interval $I$*

$$\operatorname*{in}^{I}_{\tau \in \mathcal{L}(\mathcal{M}) \cap \overline{\mathcal{L}(\mathcal{B})}} Pr^{\mathcal{M}}(\pi \in \mathcal{L}(\mathcal{A}) \mid \pi \in \mathrm{Cyl}(\mathrm{Paths}(\tau)))$$

## D Outline

See fig. 8 for an outline of the algorithm.

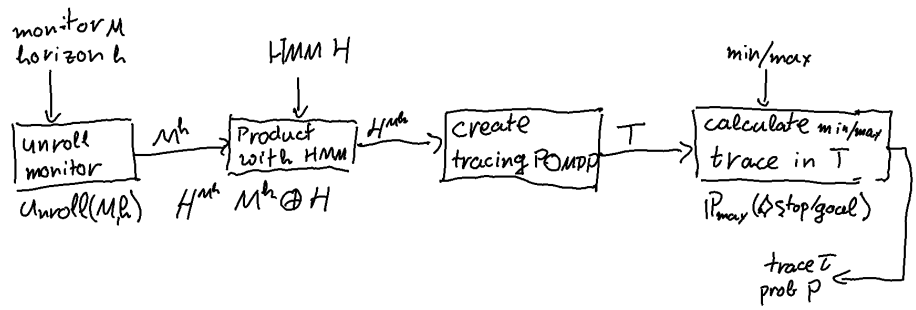



Fig. 8: Outline of the algorithm.

## E Algorithm

Our algorithm will proceed in four steps. First, we take the MC product [5] of the MC $\mathcal{M}$ and the specification $\mathcal{A}$. Then, we will combine the product MC with the monitor into an MDP. Next we will add end transitions to this MDP for all states that are accepted by the underlying monitor. Lastly, we use Paynt [2] to model check the created MDP.

### E.1 Applying the specification to the MC

We take the MC product as defined by Baier and Katoen [5] to create the product MC $\mathcal{M} \oplus \mathcal{A}$.

*Intuition* The created product MC is the original MC, but every good state is labeled.

### E.2 Constructing the product MDP

To create the MDP, we first create its states. The states are the product of the states of the product MC and the monitor. These are denoted as $\langle s, q_{\mathcal{B}} \rangle$ for $s \in S_{\mathcal{M} \oplus \mathcal{A}}$, and $q_{\mathcal{B}} \in Q_{\mathcal{B}}$. As actions in the product MDP we take the observations $C$ of the MC $\mathcal{M}$.

The transitions in the MDP are defined here in two steps. First we create a not-yet-normalized transition function $P'$ on the states in the MDP. Next, we normalize the transition function by creating redirecting all remaining probability mass of an action in a state towards the initial state in $P''$.

$$P'(\langle s, q \rangle, c, \langle s', q' \rangle) = \begin{cases} P(s, s') & \text{if } obs(s') = c \wedge q' = \delta(q, c) \\ 0 & \text{otherwise} \end{cases}$$

$$P''(\langle s, q \rangle, c, \langle s', q' \rangle) = \begin{cases} 1 - \sum_{s'' \in S} \sum_{q'' \in Q} P'(\langle s, q \rangle, c, \langle s'', q'' \rangle) & \text{if } s' = \iota_{\mathcal{M} \oplus \mathcal{A}} \wedge q' = \iota_{\mathcal{B}} \\ P'(\langle s, q \rangle, c, \langle s', q' \rangle) & \text{otherwise} \end{cases}$$

Lastly, the atomic propositions are $\{\circledcirc_{\mathcal{A}}, \circledcirc_{\mathcal{B}}\}$, where accepting states in the product MC are labeled by $\circledcirc_{\mathcal{A}}$ and accepting states in the monitor are labeled by $\circledcirc_{\mathcal{B}}$.

*Intuition* The created MDP can be seen as the original product MC $\mathcal{M} \oplus \mathcal{A}$, where we can choose which observation to observe, but only if the monitor contains the transition labeled by the observation.

### E.3 Adding end transitions

We add two new states to the MDP, the *goal* and *stop* states, each only having a self loop. Next, we will add an *end* action to every state with the label $\circledcirc_{\mathcal{B}}$. If the state also contains the $\circledcirc_{\mathcal{A}}$ label, we direct the *end* action to the *goal* state, otherwise we direct the *end* action to the stop state.

*Intuition* Since we are talking about finite traces in the monitor, we need to give the MDP a way to end a path. This is done by taking the *end* action. If we take the end action in a good state of the product MDP, we reach the *goal* state, if it was a bad state, then we reach the *stop* state.

### E.4 Model checking using Paynt

We create a POMDP which will be model checked by Paynt [2] for reachability of the goal state. The observations in this POMDP are the states of the monitor. Using this POMDP we calculate

$$\mathbb{P}_{\max}[\diamond goal]$$

to find the maximum probability and

$$1 - \mathbb{P}_{\max}[\diamond stop]$$

to find the minimum probability. Finally, we can verify this upper and lower bound with the interval $I$.

*Intuition* In order to find the traces in the monitor which give the maximum and minimum probability of reaching a good state, we associate a trace $\tau$ with a scheduler $\sigma_\tau$ in an MDP. Such a scheduler is called consistent.

**Definition 19.** *A* consistent scheduler *is a scheduler which generates only one trace*

$$\exists_{\tau \in \Sigma_{obs}} \ \forall_{\pi \in Paths(\sigma)} \ obs(\pi) = \cdots \iota \tau e^\star$$

*where $\iota$ is the observation of the initial state and $e$ is the observation of the goal and stop state.*

By model checking the created POMDP, we don't yet ensure that every scheduler is consistent, instead we only ensure semi-consistence.

**Definition 20.** *A semi-consistent scheduler $\sigma$ is a scheduler which has the following property*

$$\exists_{\tau \in \Sigma_{obs}} \ \forall_{\pi \in Paths(\sigma)} \ obs(\pi) = (\iota \tau^+)^\star$$

*where $\tau^+$ can be a different prefix of $\tau$ for every loop in $()^\star$.*

Thus, a semi-consistent scheduler does not have to take and end action. However, by maximizing the probability of reaching either the *goal* or *stop* state, we ensure that if the *end* action can be taken, it will be taken. And if a path takes the *end* action, almost surely any other path will also take the *end* action at some point. This property ensures that when maximizing semi-consistent schedulers we will always get a consistent scheduler, and thus a single trace.

## References


[1] Roman Andriushchenko et al. "Inductive synthesis of finite-state controllers for POMDPs". In: *UAI*. Vol. 180. Proceedings of Machine Learning Research. PMLR, 2022, pp. 85–95.

[2] Roman Andriushchenko et al. "PAYNT: A Tool for Inductive Synthesis of Probabilistic Programs". In: *Computer Aided Verification*. Ed. by Alexandra Silva and K. Rustan M. Leino. Cham: Springer International Publishing, 2021, pp. 856–869. ISBN: 978-3-030-81685-8. DOI: 10.1007/978-3-030-81685-8_40.

[3] Dana Angluin. "Learning Regular Sets from Queries and Counterexamples". In: *Inf. Comput.* 75.2 (1987), pp. 87–106.

[4] Thom S. Badings et al. "CTMCs with Imprecisely Timed Observations". In: *TACAS (2)*. Vol. 14571. Lecture Notes in Computer Science. Springer, 2024, pp. 258–278.

[5] Christel Baier and Joost-Pieter Katoen. *Principles of Model Checking*. MIT Press, 2008. ISBN: 978-0-262-02649-9.

[6] Christian Hensel et al. "The Probabilistic Model Checker Storm". In: *International Journal on Software Tools for Technology Transfer* 24.4 (Aug. 1, 2022), pp. 589–610. ISSN: 1433-2787. DOI: 10.1007/s10009-021-00633-z. URL: https://doi.org/10.1007/s10009-021-00633-z (visited on 09/02/2024).

[7] Orna Lichtenstein, Amir Pnueli, and Lenore Zuck. "The Glory of the Past". In: *Logics of Programs*. Ed. by Rohit Parikh. Berlin, Heidelberg: Springer, 1985, pp. 196–218. ISBN: 978-3-540-39527-0. DOI: 10.1007/3-540-15648-8_16.

[8] Frits W. Vaandrager. "Model learning". In: *Commun. ACM* 60.2 (2017), pp. 86–95.